

Pulse — Free-Tier Deployment Guide (No Credit Card)

This document is written for an AI coding agent (Cursor / Claude Code / Antigravity) to execute deployment for the Pulse project (API Reliability & Incident Management Platform). Follow the steps in order. Do not skip ahead.

Goal: Deploy the full stack (Next.js frontend, Express backend, Temporal worker + server, MongoDB, Redis) entirely on free tiers with no credit card, and keep it live 24/7 via uptime pinging so it's demoable to interviewers at any time.

0. Prerequisites

- Repo already built per the PRD (`apps/frontend` , `apps/backend` , `packages/shared-types`).
 - GitHub account with the repo pushed.
 - Accounts needed (all free, no card): MongoDB Atlas, Upstash, Render, Vercel, cron-job.org.
-

1. Database — MongoDB Atlas (Free M0)

1. Sign up at <https://www.mongodb.com/cloud/atlas/register>
 2. Create a free **M0 cluster** (shared tier, no card required).
 3. Create a database user (username/password) — save these.
 4. Network Access → Add IP Address → **Allow Access from Anywhere** (0.0.0.0/0), since Render's IPs aren't static on the free tier.
 5. Get the connection string:

```
mongodb+srv://<user>:  
<password>@<cluster>.mongodb.net/pulse?  
retryWrites=true&w=majority
```
 6. Save this as `MONGO_URI`.
-

2. Cache / Rate Limiting — Upstash Redis (Free)

1. Sign up at <https://upstash.com> (no card required for free tier).
2. Create a new Redis database (choose a region close to your Render region for lower latency).

3. Copy the **Redis connection URL** (rediss:// TLS URL).
 4. Save this as `REDIS_URL` .
-

3. Temporal Server — Render Free Web Service

Temporal needs its own always-reachable server. Self-host it on Render rather than using Temporal Cloud (which typically requires a card).

1. In the repo, confirm `docker-compose.yml` has a working `temporalio/auto-setup` service definition — reuse that image config for the Render service.
2. On Render (<https://render.com>, no card required):
 - New → Web Service → connect GitHub repo (or use "Deploy an existing image" with `temporalio/auto-setup:latest`).
 - Set required env vars per the Temporal auto-setup image docs (points it at a Postgres/SQLite persistence store — for free-tier simplicity, use Temporal's built-in SQLite dev mode if supported by the image version, otherwise provision a small free Postgres too).

- Expose the Temporal frontend port and add a lightweight `/healthz` -equivalent check (Temporal has its own health endpoint on port 7233; if Render's free web service requires HTTP health checks, add a tiny sidecar/health route or use the Temporal Web UI port as the health-checked port).

3. Note the internal Render URL — this becomes `TEMPORAL_ADDRESS` for the worker and backend.

4. Backend API — Render Free Web Service

1. New Web Service on Render → connect repo → root directory `apps/backend`.
2. Build command: `npm ci && npm run build -w apps/backend`
3. Start command: `node apps/backend/dist/server.js` (adjust to actual build output path).
4. Environment variables (from `.env.example` in the PRD):
 - `MONGO_URI` — from step 1
 - `REDIS_URL` — from step 2
 - `JWT_SECRET` — generate a random 32+ char string

- `TEMPORAL_ADDRESS` — from step 3
 - `ANTHROPIC_API_KEY` — your Claude API key
 - `RESEND_API_KEY` — from Resend (free tier, no card, for transactional email)
 - `ENCRYPTION_KEY` — random 32-byte key for AES-256-GCM (authToken encryption)
 - `NODE_ENV=production`
5. Confirm `/healthz` and `/readyz` routes exist and return 200 — Render uses these for its own health checks too.
6. Deploy. Note the public URL, e.g. `https://pulse-backend.onrender.com`.
-

5. Temporal Worker — Render Free Web Service (separate from API)

Run the worker as its own service so a crash/restart doesn't take down the API.

1. New Web Service on Render → same repo → root directory `apps/backend`.
2. Start command: `node apps/backend/dist/temporal/worker.js` (adjust to actual build output).
3. Same env vars as step 4 (it needs `MONGO_URI`, `REDIS_URL`, `TEMPORAL_ADDRESS`,

`ANTHROPIC_API_KEY`).

4. **Important:** Render free web services require an HTTP port to bind to and respond on, even for a "worker." Add a minimal Express route (e.g. `/healthz` returning `200 OK`) inside the worker process purely so Render treats it as a healthy web service and so the uptime pinger (step 8) has something to hit.
-

6. Frontend — Vercel

1. Sign up at <https://vercel.com> (no card required for Hobby tier).
 2. Import the GitHub repo → set root directory to `apps/frontend` .
 3. Framework preset: Next.js (auto-detected).
 4. Environment variables:
 - `NEXT_PUBLIC_API_URL` — the Render backend URL from step 4.
 5. Deploy. Vercel's Hobby tier doesn't sleep — this stays live with no pinging needed.
-

7. Wire Up CORS & URLs

- In the backend, allow CORS from the Vercel frontend domain.
 - In the frontend, confirm all API calls use `NEXT_PUBLIC_API_URL`.
 - Redeploy both after confirming URLs match.
-

8. Keep Render Services Alive — cron-job.org (Free, No Card)

Render's free tier sleeps a service after 15 minutes of no HTTP traffic. To keep everything live for on-demand interviewer demos:

1. Sign up at <https://cron-job.org> (free, no card).
2. Create 3 cron jobs, each running every **10 minutes**:
 - GET `https://pulse-backend.onrender.com/healthz`
 - GET `https://pulse-worker.onrender.com/healthz`
 - GET `https://pulse-temporal.onrender.com/<health-endpoint>`
3. Save and enable all three.

This keeps all Render services warm 24/7 without paying anything.

9. Seed Demo Data

Per PRD milestone M9: run the seed script against the production `MONGO_URI` to create:

- A demo team + user (with known login credentials to show interviewers).
- An intentionally flapping demo endpoint, so an incident can be triggered live during a demo.

10. Pre-Interview Checklist

Run this 30 minutes before any interview where you'll demo live:

- ☐ Open the public status page cold — confirm it loads (catches Render cold-start issues even with pinging).
- ☐ Log in with demo credentials.
- ☐ Confirm the flapping demo endpoint has recently triggered an incident (or manually trigger one).
- ☐ Check Mongo Atlas and Upstash dashboards for any free-tier connection limit warnings.
- ☐ Confirm all 3 cron-job.org pings show "successful" in their last-run history.

Known Free-Tier Limitations (be upfront about these if asked)

- MongoDB Atlas M0: 512 MB storage cap — fine for a demo, not for real traffic.
- Upstash free tier: request-count cap per day.
- Render free web services: 750 combined hours/month across all your free services — running 3 services 24/7 will consume this faster than one; monitor usage.
- No SLA — free tiers can have occasional cold starts or brief downtime despite pinging.